



Laboratoire d'Études Spatiales et d'Instrumentation en Astrophysique

Iut d'Evry
2^e année, BUT GEII,
Parcours Electronique et
Systèmes embarqués

Tuteur école :
Jérôme DELPERIE

Observatoire de Paris
Laboratoire d'Études Spatiales et
d'Instrumentation en
Astrophysique
Meudon, France
Tuteur entreprise :
Moustapha DEKKALI

Rapport de stage

Réalisation d'une interface visuelle afin de modéliser les signaux entrants

Du 13 Janvier au 7 Mars 2025

Fares BELBACHIR

LIRA



**Observatoire
de Paris**

PSL



Remerciements

À l'issue de ce stage, je tiens à remercier toutes les personnes qui ont contribué au succès de mon stage. Je tiens à leur témoigner toute ma gratitude pour leur temps et leur disponibilité.

J'adresse mes remerciements, dans un premier temps, à tout le personnel du Service Ingénierie électronique du Lira (SIEL) pour leur accueil chaleureux.

Tout particulièrement, je tiens à remercier **M. Masood SHERYAR**, ingénieur informatique, pour son encadrement, ses conseils avisés et le temps qu'il m'a consacré tout au long de ce stage ainsi que **M. Pierre-Luc Astier**.

Je tiens à remercier **M. Videch Gamage** notamment pour ses explications théoriques et concrètes autour des FPGA.

Pour finir, je tiens à remercier toutes les personnes qui m'ont conseillé et relu lors de la rédaction de ce rapport de stage notamment mon tuteur, **M. Moustapha DEKKALI**, responsable du SIEL.

Table des matières

1. Introduction	4
2. Présentation de l'entreprise.....	5
2.1 Observatoire de Paris et le site de Meudon	5
2.2 Le Laboratoire d'Instrumentation et de Recherche en Astrophysique.....	7
3. Le projet et la mission	10
3.1 La familiarisation avec Python.....	10
3.2 Objectif	10
3.3 . Acquisition des données sur Arduino	13
3.4 Création de l'interface sur Pyqt 6.....	19
3.5 Conception du code Python.....	23
4. Conclusion	31
5. Annexes	32
6. Liste des abréviations et sigles employés.....	37
7. Bibliographie	38
8. Crédits iconographiques.....	39

Abstract en anglais

As part of my internship in computer science, I was assigned the task of implementing a test kit supporting a high-resolution ADC and developing a Python interface to control it and graphically display the data. These tools are intended for users to enable them to visualize the signals and analyze the performance of this component. This internship occurred at the Paris Observatory, specifically at the Meudon site, from January 13 to March 7, 2025, over a period of 8 weeks.

The Paris Observatory, established in 1667, ranks among the oldest and most esteemed scientific institutions globally. Situated in Paris, the observatory is pivotal in the fields of astronomy, physics, and broader universe sciences. The institution carries out advanced research across diverse domains, including cosmology, astrophysics, and Earth sciences, while simultaneously functioning as a hub for the education and training of researchers and students.

The objective was to develop a graphical interface in Python, using PyQt 6 (a tool for interface development), to visualize real-time analog signals, alongside Visual Studio Code (a code editor) for Python programming. The goal was to create an interface that was both simple and functional, allowing for the display of dynamic curves while incorporating interactive features such as zooming, panning, and the primary function of selecting the number of samples to visualize from the signal.

One of the principal challenges faced was managing values due to the limited RAM capacity of the computer, as well as performance management difficulties when displaying dense signals without inducing lag in the interface. Additionally, the implementation of user interactivity, especially in managing events such as zooming or region selection, necessitated expertise in asynchronous events and a strong proficiency in Python. The precise representation of analog signals and the optimization of real-time display posed technical hurdles, requiring meticulous data processing and the management of the most recent graphical updates. Completing these tasks within the 8-week timeframe proved to be a considerable challenge.

The result is an interface where the user controls the number of samples used to visualize the signal emitted by the waveform generator. An increased sample count translates to enhanced precision of the signal. The interface is equipped with two scroll bars (one vertical and one horizontal) for comprehensive navigation of the full signal. This internship enabled me to cultivate extensive technical skills, particularly in Python programming, encompassing concepts such as classes, data structures, file manipulation, and the creation of graphical interfaces utilizing PyQt 6.

From a human perspective, this internship offered a fulfilling experience in teamwork, collaboration with industry professionals, and the sharing of knowledge, which significantly enhanced my technical communication abilities. Professionally, it bolstered my capacity to address complex problems, optimize performance, and devise interactive solutions tailored to user requirements. Additionally, this internship deepened my understanding of the prerequisites involved in a software development project, from conception through implementation, while nurturing my diligence and adaptability when confronting technical challenges. This internship opened my vision a little more to the different professions around embedded systems. I plan to continue my studies in this field and specialize in embedded systems.

1 Introduction

Dans le cadre de ma deuxième année d'étude à l'IUT d'Evry en BUT GEII (Génie électrique et informatique industrielle), j'ai réalisé un stage en entreprise de 8 semaines, essentiellement axé sur le développement informatique mais en interface avec du matériel. Mes recherches se sont tournées vers un stage de développement logiciel, car je souhaitais découvrir ce métier. J'ai effectué mon stage du 13 Janvier 2025 au 7 Mars 2025 au Le **LIRA** (Laboratoire d'Instrumentation et de Recherche en Astrophysique) de l'Observatoire de Paris, sur le site de Meudon. Plus exactement, mon stage s'est déroulé au sein du Service d'Ingénierie Electronique du LIRA (SIEL).

J'ai choisi cet organisme pour son secteur d'activité (instrumentation pour l'astrophysique) et pour son expertise technique, notamment en électronique et en informatique. De plus, la mission proposée correspondait parfaitement à mes attentes.

Le LIRA est un laboratoire spécialisé dans le domaine de l'astrophysique. Il a pour vocation de développer et exploiter des instruments d'observation au sol (grands télescopes) et embarqués à bord de sondes spatiales opérées par les grandes agences internationales. Ces instruments permettent l'acquisition de données, futures bases pour le travail des chercheurs. A ce titre, le LIRA dispose d'une expertise reconnue en instrumentation qu'il développe constamment.

Dans ce contexte, mon stage portait sur deux tâches principales :

- La mise en œuvre d'un kit comprenant un ADC (Convertisseur Analogique/Numérique) à haute résolution
- Et le développement d'une interface Python pour le contrôler le kit et afficher graphiquement les données issues de cet ADC

La finalité de l'outil que j'ai développé est de tester cet ADC et de mesurer finement ses performances afin d'évaluer son adéquation avec les futurs projets

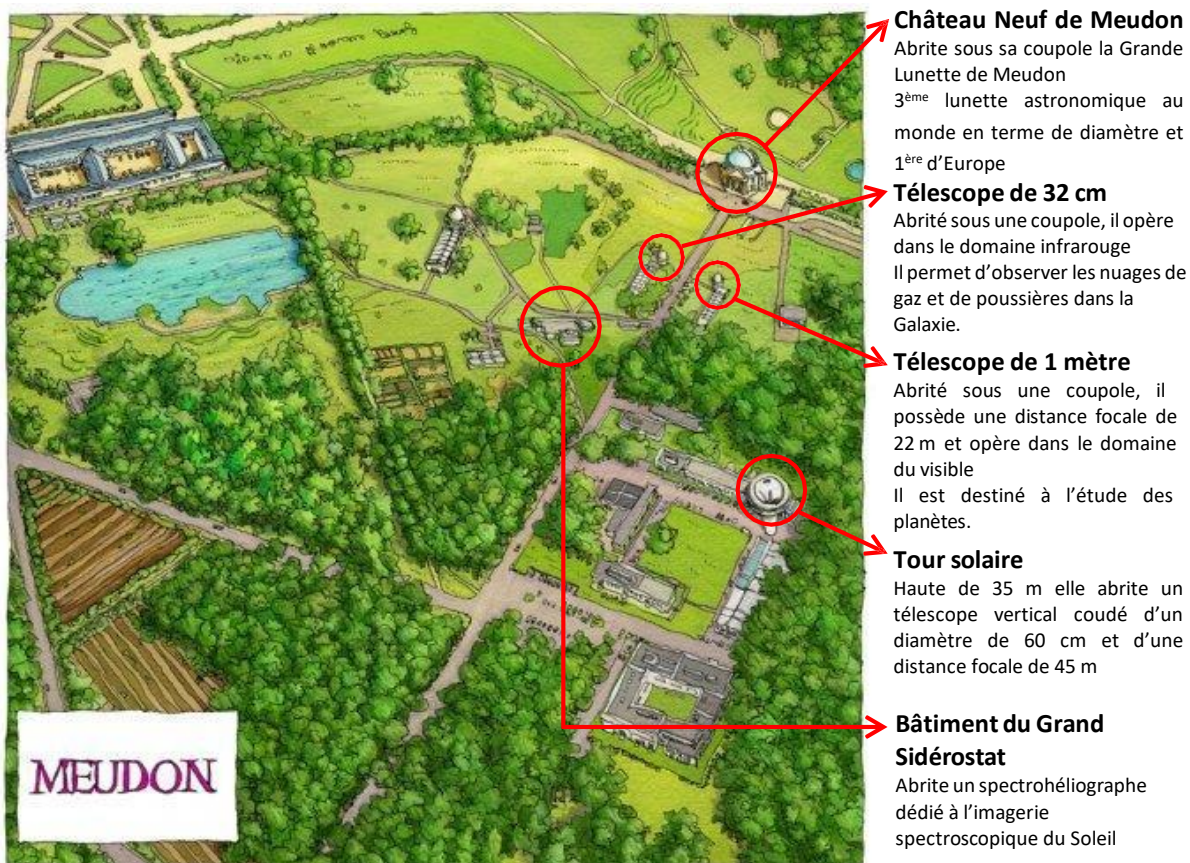
Dans un premier temps, je décrirai l'Observatoire de Paris et son histoire ; puis le LIRA, ses missions et les projets dans lesquels il est impliqué. Ensuite, je détaillerai les missions qui m'ont été confiées durant ce stage et les travaux que j'ai mené. Je décrirai notamment l'interface graphique que j'ai conçue et qui permet d'afficher le signal issu d'un générateur de fonction qui aura préalablement été numérisé. Pour finir, je dresserai un bilan de cette expérience.

2 Présentation de l'entreprise

2.1 Observatoire de Paris et le site de Meudon

Fondé en 1667 à Paris, l'Observatoire de Paris est un grand établissement national de recherche en astronomie créé pour accueillir les astronomes académiciens du Roi Soleil. Deux siècles plus tard, en 1876, l'Observatoire de Meudon est créé à l'initiative de l'astronome français Jules Janssen avant d'être rattaché à la structure de Paris en 1926. Ce site sera dédié à l'astrophysique. En 1953, la station de radioastronomie de Nançay est également rattachée à l'Observatoire de Paris qui compte désormais trois sites de recherche.

Durant ses 350 ans d'existence, l'Observatoire de Paris va se doter d'instruments novateurs et performants. De nos jours sur le site de Meudon, les observations nocturnes ont cessé mais des observations quotidiennes du Soleil sont encore réalisées et ce lieu accueille de nombreux astrophysiciens.



Panorama du site de Meudon

Au fil des années, des découvertes majeures, toujours d'actualité, ont été faites au sein de chacun des trois sites de l'Observatoire de Paris. Nous pouvons citer notamment la découverte de quatre satellites de Saturne en 1671, la découverte de la planète Neptune en 1846, l'invention du coronographe en 1930 qui permet de fabriquer des éclipses de soleil artificielles, ou encore la mise en place de l'optique adaptative en 1993.

L'Observatoire de Paris remplit trois missions principales qui sont :

- **La recherche**, en contribuant au progrès de la connaissance de l'Univers,
- **La formation initiale et continue** au sein de ses locaux ou dans des universités partenaires (l'Observatoire de Paris relève du Ministère de l'Enseignement supérieur et de la Recherche),
- **La diffusion des connaissances** au grand public via des journées portes ouvertes, expositions, etc.

À ce jour, l'Observatoire de Paris est le plus grand pôle national et européen de recherche en astronomie et le plus ancien observatoire au monde à être encore en activité. 30 % des astronomes français y poursuivent leurs recherches au sein de cinq laboratoires et un institut regroupé dans le pôle « Départements scientifiques ».

Organigramme de l'Observatoire de Paris en Décembre 2024

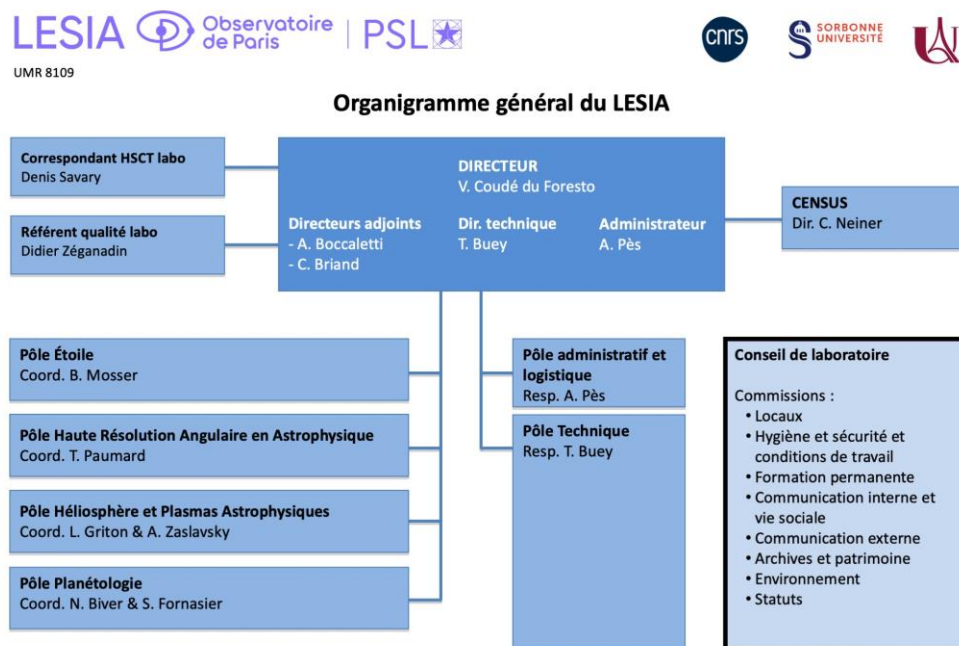
Depuis cette année, l'Observatoire de Paris s'est entièrement restructuré. Cependant, l'organigramme n'a pas encore été mis à jour.

2.2 Le LIRA – Laboratoire d’Instrumentation et de Recherche en Astrophysique

Le LIRA est un des départements de l’Observatoire de Paris implanté sur le site de Meudon. Il est également une Unité Mixte de Recherche (UMR) du CNRS. Ce laboratoire est le plus gros laboratoire français de recherche en astrophysique représentant 12 % du personnel dans la discipline. Début 2018, il était composé de 242 agents dont 132 permanents répartis entre des chercheurs/enseignants-chercheurs (65 personnes) et des ingénieurs-techniciens-administratifs (67 personnes).

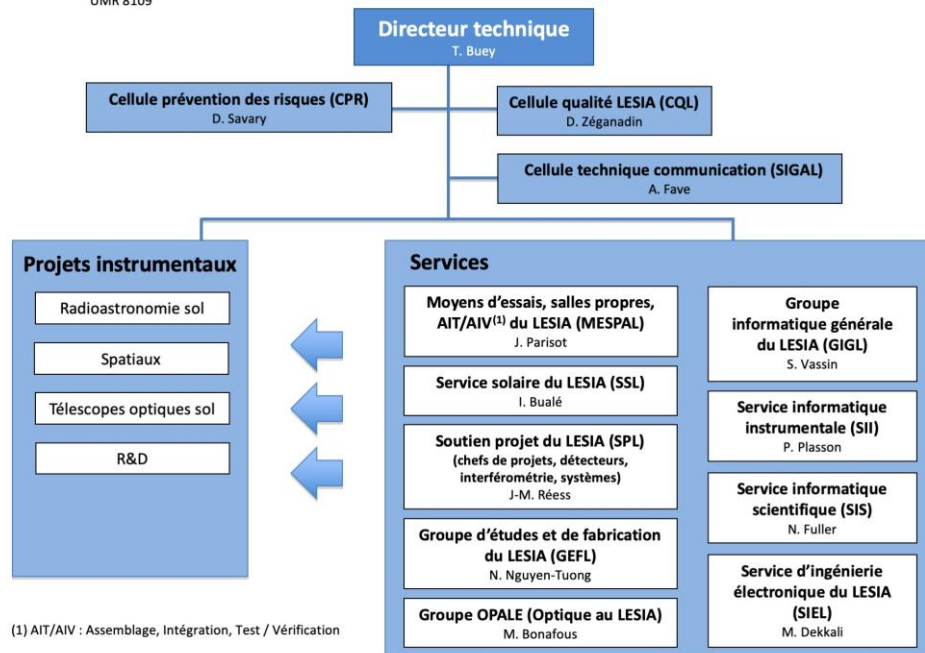
Le LIRA a pour vocation première de concevoir et de réaliser des instruments scientifiques spatiaux et sols. Il est également responsable de l’exploitation et de l’interprétation scientifique des observations réalisées par ces instruments. De plus, le laboratoire s’engage sur le développement de techniques avancées mises en œuvre dans des instruments au sol ainsi que des instruments spatiaux.

Les activités du LIRA sont structurées en cinq pôles scientifiques : Étoile, Haute Résolution Angulaire en Astrophysique, Physique des Plasmas, Physique Solaire et Planétologie. De plus, il comprend une administration et un important pôle technique auquel je suis rattachée.



Organigramme général du LESIA en avril 2024 (LIRA depuis janvier 2025)

Le pôle technique est structuré en plusieurs services spécialisés. Dans le cadre de mon stage, j’ai intégré le SIEL (Service d’Ingénierie Electronique du LIRA) qui regroupe les techniciens et ingénieurs spécialisés dans les métiers de l’électronique.



Organigramme général du pôle technique du
LESIA (Juin 2017)

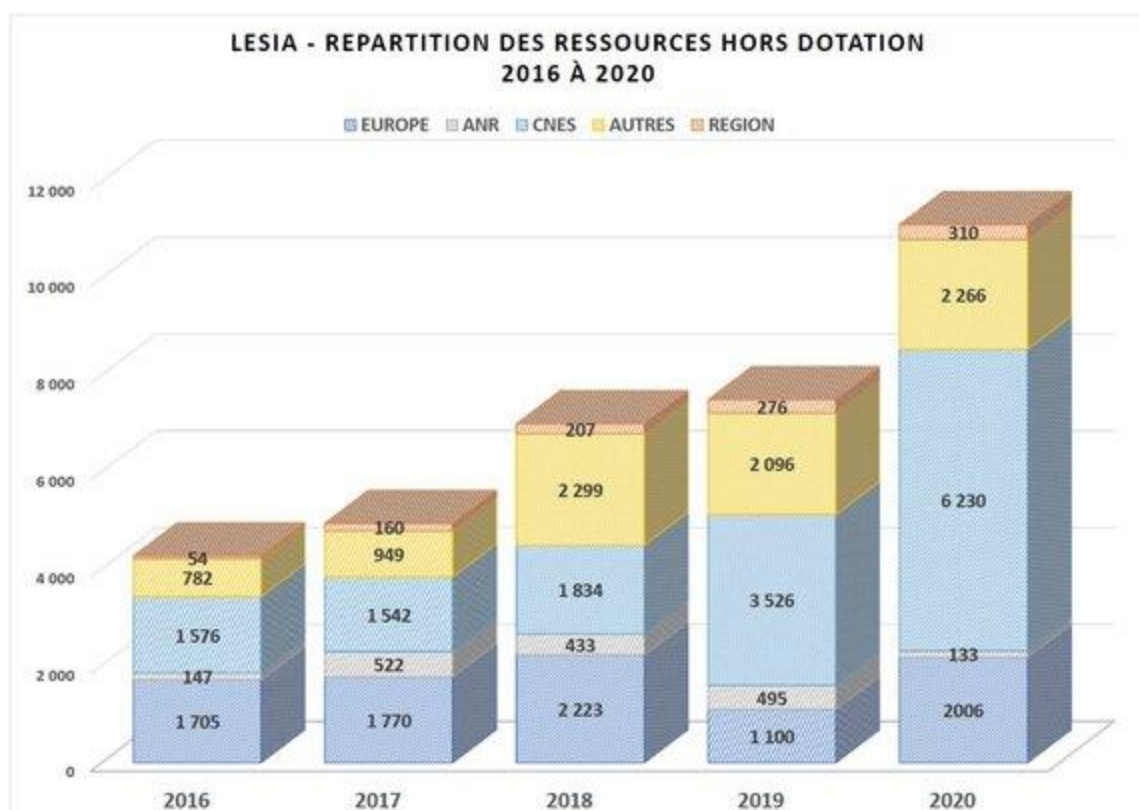
*LESIA ancien nom du LIRA

Depuis sa création, le laboratoire a eu l'opportunité de travailler sur plusieurs projets d'envergure. Parmi les plus récents, on peut citer :

- L'instrument VIRTIS, un spectro-imageur visible et proche infrarouge sur la mission Rosetta, pour l'étude de la comète 67P/Tchourioumov- Guérassimenko
- L'instrument SORBET, un analyseur d'ondes radio sur la mission BepiColombo pour l'étude de la petite magnétosphère de Mercure et son interaction avec le vent solaire
- La voie infrarouge de l'instrument SuperCam sur le rover Perseverance qui permet d'analyser le sol martien
- L'instrument Gravity sur le Very Large Telescope (VLT au Chili) qui est capable de sonder l'espace-temps jusqu'à la frontière du trou noir super-massif, Sagittarius A, qui occupe le cœur de notre galaxie, la Voie Lactée

Le laboratoire est également très impliqué dans de nombreux aspects des techniques et technologies de l'information, pour le développement des méthodes instrumentales et pour l'analyse de données.

Enfin pour le budget du laboratoire, il provient au $\frac{3}{4}$ de ressources propres et au $\frac{1}{4}$ de subventions de l'État. Les ressources propres proviennent de plusieurs origines : la plus importante est issue du CNES. Les autres sources de financement sont l'ANR, l'Europe, l'INSU, la Région Île-de-France pour ne citer que les plus importants.



3 Le projet et la mission

3.1 La familiarisation avec Python

Le projet a débuté par une phase de familiarisation avec le langage Python, visant à acquérir les concepts essentiels de la programmation. Cela comprend l'apprentissage de structures de données essentielles comme les listes, les dictionnaires et les sets, permettant de mieux organiser et manipuler les informations. J'ai également exploré la gestion des exceptions, en particulier l'utilisation de «.format » pour le formatage des chaînes de caractères, afin de rendre le code plus robuste. En parallèle, j'ai développé des compétences en manipulation de fichiers, facilitant l'interaction avec des données externes. Enfin, la compréhension des principes de l'héritage et des classes m'a permis d'aborder la programmation orientée objet, offrant une meilleure organisation et réutilisation du code.

3.2 Objectif

L'objectif de mon projet est de mettre en œuvre le kit ADC et de créer une interface en Python pour collecter et visualiser les échantillons numériques issus d'un signal analogique. Le schéma fonctionnel du banc est donné ci-dessous.

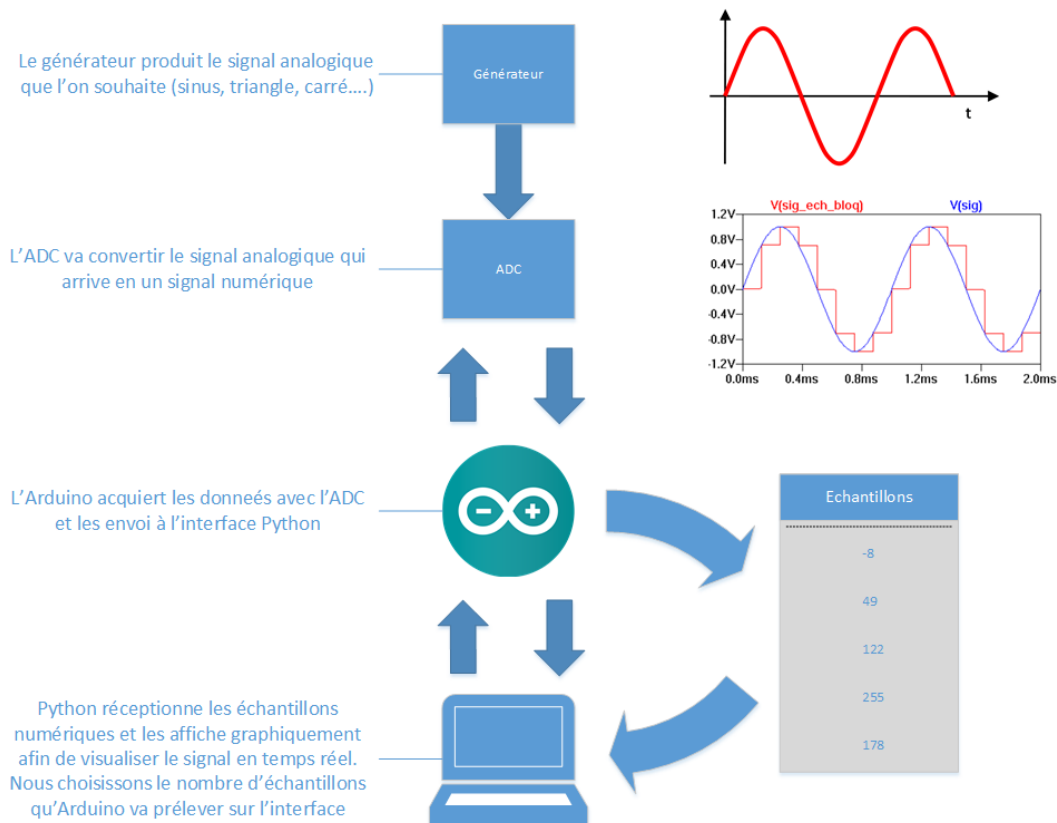


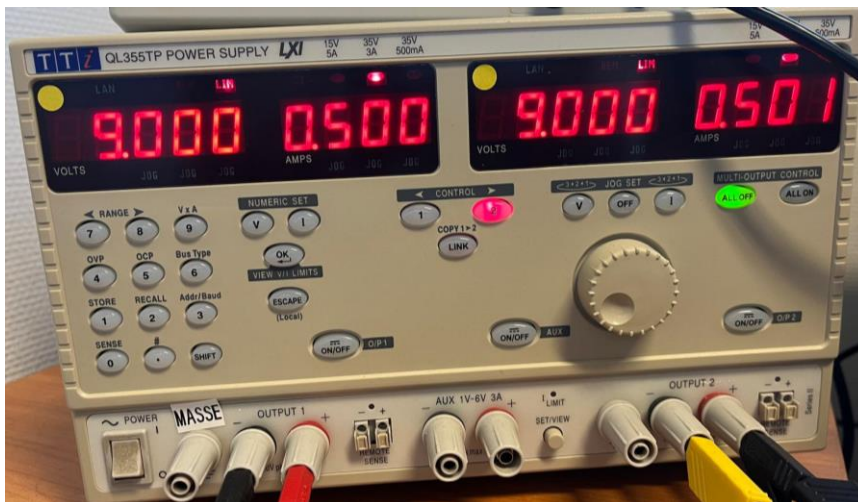
Schéma 1 – Schéma fonctionnel du projet

Le générateur produit un signal analogique qui est injecté dans l'ADC qui le converti en un signal numérique. La carte Arduino assure l'interface entre l'ADC et le PC supportant l'application Python. Il permet de prélever un certain nombre d'échantillons du signal à une fréquence définie puis il transmet ces données numériques via une connexion série vers l'interface Python. L'application Python reçoit les échantillons qu'elle stocke dans un tableau avant de les afficher graphiquement. Ceci permet de visualiser un signal en temps réel.

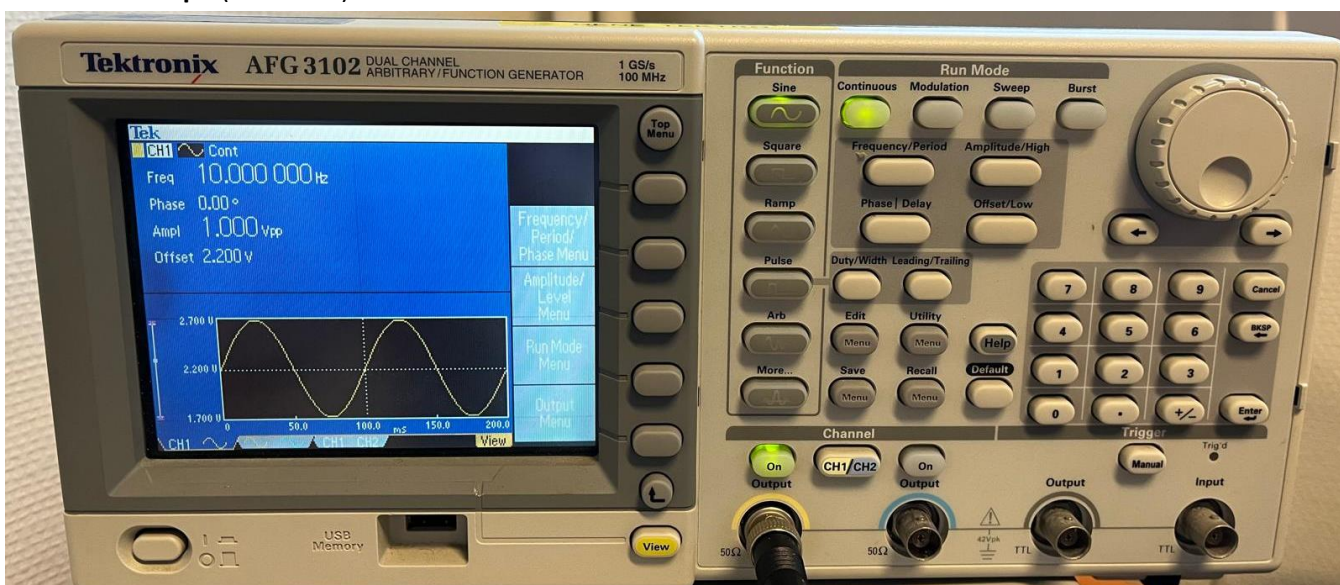
Matériel utilisé :

Pour le matériel utilisé je me suis servi :

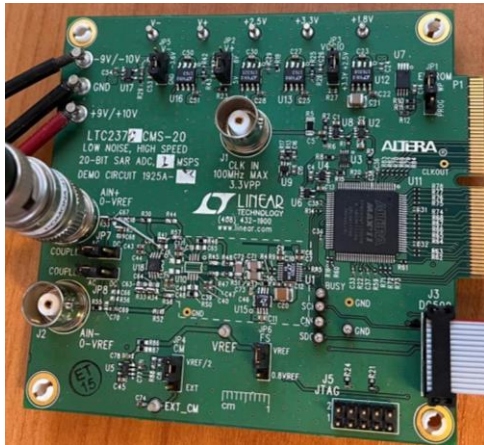
- d'un **générateur de signaux** (QL355TP) : Il produit un signal analogique (ex. : sinusoïdal, carré, ...)



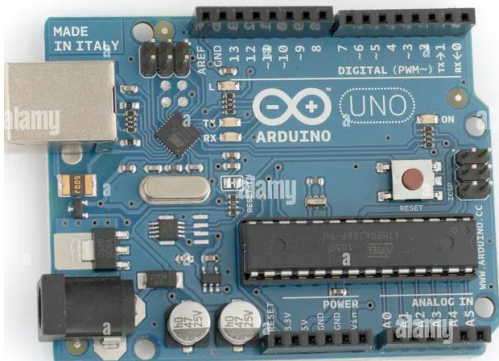
- d'un **oscilloscope** (Tektronix)



- d'un **convertisseur analogique-numérique, un ADC (LTC 237)** de haute précision permettant de convertir un signal analogique en une valeur numérique, avec une haute précision et une faible consommation d'énergie sont essentielles. Il fonctionne entre -9V et +9V (différent des valeurs de la doc technique qui est en annexe car les ingénieurs l'ont modifié).



- et d'une **carte Arduino** (documentation technique en annexe). Elle contrôle l'ADC, récupère les données numériques et peut envoyer ces données à un PC via une communication série (une RS232).



□

Câblage :

Le câblage commence par connecter le générateur de signaux à l'oscilloscope pour visualiser le signal analogique. Ensuite, le signal analogique de l'oscilloscope est envoyé à l'entrée J1 de l'ADC LTC237 qui va convertir ce signal analogique en numérique, dans une plage de -9V à +9V. L'ADC est relié à la carte Arduino via l'interface SPI (Serial Peripheral Interface), avec des connexions pour le MISO (Master In Slave Out), SCK (Serial Clock), et CS (Chip Select), permettant à l'Arduino de récupérer les données numérisées. Enfin, l'Arduino envoie ces données numériques au PC via une connexion RS232 ou un adaptateur USB-to-RS232 pour la communication série, permettant ainsi d'afficher ou d'analyser les données sur le PC.

3.3 Acquisition des données sur Arduino

La première étape est de **lire la data** sur Arduino. Lorsque l'on parle de "lire la data" dans ce contexte, cela signifie que l'Arduino reçoit des signaux ou des informations brutes d'un capteur, les interprète, et les transforme en données compréhensibles pour l'utilisateur. Pour cela, il faut faire un code Arduino qui interagisse avec l'ADC, via l'interface SPI afin de collecter les échantillons de données.

Tout d'abord je définis quelques constantes et variables pour configurer et lire des données à partir d'un convertisseur analogique-numérique (ADC), ici le LTC237 via l'interface SPI.

La broche 48 (nBusyPin) est utilisée pour vérifier si le LTC237 est occupé ou non en traitant une opération, et la broche 10 (chipSelectPin) est la broche de sélection du chip qui permet de communiquer avec le LTC237.

La **fréquence d'échantillonnage** est définie à 100 Hz, ce qui signifie que l'Arduino va acquérir des échantillons à cette fréquence.

La variable nbrSamples (nombre d'échantillons) est initialisée à 10, mais pourra être ajustée dynamiquement, et adcSamples est un tableau destiné à stocker les valeurs des échantillons obtenus, avec une capacité maximale de 8191 échantillons (le nombre maximum autorisé).

Ce code constitue une base pour configurer l'acquisition de données à partir du LTC237 via l'interface SPI.

```
#include <SPI.h>

const int nBusyPin = 48;           //Définit la broche utilisée pour vérifier si le LTC237 est occupé
const int chipSelectPin = 10;      // Pin de sélection du LTC237
const int sampleRate = 100;        // Fréquence d'échantillonnage en Hz
int nbrSamples = 10;               // Valeur par défaut du nombre d'échantillons
long adcSamples[8191];             // Tableau pour stocker les échantillons (max 8191)
```

Ici, j'initialise la communication de l'Arduino avec un PC et configure l'interface SPI pour la communication avec le LTC237. La fonction `Serial.begin(115200)` démarre la communication série à une vitesse de 115200 bauds, permettant à l'Arduino d'envoyer des données au PC.

`SPI.begin()` initialise l'interface SPI, nécessaire pour la communication avec des périphériques comme le LTC237. Ensuite, la broche de sélection du chip (CS, chipSelectPin) est définie comme une sortie avec `pinMode(chipSelectPin, OUTPUT)` et est initialement mise à HIGH avec `digitalWrite(chipSelectPin, HIGH)`, ce qui désactive le LTC237 (la valeur HIGH sur cette broche indique généralement que le périphérique n'est pas sélectionné).

Enfin, un message est envoyé au moniteur série pour indiquer que l'Arduino est prêt et expliquer comment modifier le nombre d'échantillons via la commande "samples <number>".

```
// Initialiser l'interface SPI
void setup() {
  Serial.begin(115200); // Communication série avec le PC
  SPI.begin();          // Initialisation de l'interface SPI
  pinMode(chipSelectPin, OUTPUT); // Définir la pin CS comme sortie
  digitalWrite(chipSelectPin, HIGH); // Désactiver le LTC237

  Serial.println("Arduino ready. Type 'samples <number>' to change sample count.");
}
```

Cette fonction `readADC()` permet de **lire les données** de l'ADC LTC237 via l'interface SPI. La fonction commence par activer le LTC237 en mettant la broche `chipSelectPin` à LOW, puis attend que l'ADC soit prêt en vérifiant la broche `nBusyPin`. Ensuite, il y a trois transmissions SPI pour lire les trois octets (8 bits chacun) renvoyés par l'ADC. Ces octets (stockés dans `highByte`, `midByte`, et `lowByte`) représentent la valeur échantillonnée par l'ADC, qui est de 24 bits au total. Après avoir désactivé le LTC237 en mettant la broche `chipSelectPin` à HIGH, les trois octets sont assemblés en une valeur de 32 bits (via des opérations bit à bit) pour obtenir le résultat final.

□

Ce résultat est ensuite affiché dans le moniteur série et retourné par la fonction. En résumé, cette fonction permet de lire et d'assembler les **données brutes** de l'ADC en une valeur numérique 32 bits.

```
// Fonction pour lire les données du LTC237 via SPI
long readADC() { //Déclare une fonction qui lit les données de l'ADC et retourne un long (valeur 32 bits)
    long result = 0; // initialisation a 0

    // Commencer la communication SPI
    digitalWrite(chipSelectPin, LOW); // Activer le LTC237 (CS bas)
    while (digitalRead(nBusyPin) == HIGH);
    // Transmission des 16 bits de données via SPI
    byte highByte = SPI.transfer(0x00); // Demande de lecture
    byte midByte = SPI.transfer(0x00);
    byte lowByte = SPI.transfer(0x00); // Lecture du byte bas de la donnée
    //Chaque SPI.transfer envoie un octet et reçoit un octet. L'ADC renvoie un total de 24 bits pour la valeur échantillonnée
    digitalWrite(chipSelectPin, HIGH); // Désactiver le LTC237 pour indiquer que l'ADC est prêt.

    // assemble les trois octets dans un mot de 32-bit
    result = ((char)(highByte))*4096L | (midByte<<4) | (lowByte>>4);
    // maintenant, result est un nombre signé de 32-bit !

    //Serial.println( (highByte * 65536) | (midByte * 256) | lowByte );
    Serial.println( result);
    return result;
}
```

Puis, je devais faire la fonction `loop()` qui permet à l'Arduino de recevoir et d'exécuter des commandes envoyées via le terminal série. Lorsque des données sont disponibles dans le buffer série, le programme lit la commande envoyée par l'utilisateur et la vérifie.

Si la commande commence par "*samples*", cela signifie qu'on souhaite modifier le nombre d'échantillons à prendre.

Le nombre d'échantillons est extrait de la commande et converti en entier. Si ce nombre est valide (entre 1 et 8191), le programme démarre un processus d'échantillonnage, où il lit des valeurs à partir de l'ADC à l'aide de la fonction `readADC()`, **stocke ces valeurs** dans un tableau, et attend un délai défini par la fréquence d'échantillonnage avant de prendre le prochain échantillon.

Une fois l'échantillonnage terminé, le programme affiche un message de confirmation. Si le nombre

d'échantillons est invalide, il affiche un message d'erreur.

```
void loop() {
  // Si une nouvelle commande est envoyée via le terminal
  if (Serial.available() > 0) { //Vérifie si des données sont disponibles dans le buffer série
    String command = Serial.readStringUntil('\n'); // Lire la commande du terminal
    command.trim(); // Supprimer les espaces inutiles

    // Vérifier si la commande est 'samples' et change le nombre d'échantillons
    if (command.startsWith("samples")) {
      int newSamples = command.substring(8).toInt(); // Extraire la valeur du nombre d'échantillons
      if (newSamples > 0 && newSamples <= 8191) { // Vérifier si le nombre d'échantillons est valide
        nbrSamples = newSamples; //Si la valeur est valide, met à jour la variable nbrSamples avec la nouvelle valeur.

        // Echantillonnage
        for (int i = 0; i < nbrSamples; i++) { //Démarre une boucle pour lire plusieurs échantillons.
          long adcValue = readADC(); // Lire valeur ADC
          adcSamples[i] = adcValue; // Stocke la valeur lue dans le tableau
          delay(1000 / sampleRate); // Attendre avant de prendre un nouvel échantillon
        }

        Serial.println("Sampling completed.");
      } else {
        Serial.println("Invalid, CHOISIR UN NOMBRE ENTRE 1 ET 8191.");
      }
    }
  }
}
```

Quelques erreurs rencontrées :

```
Serial.print("SignalVoltage: ");
```

Dans cette ligne de code, j'ai laissé un **espace après les deux points**. Ce n'est pas une erreur en soi, mais cela n'affecte pas la fonctionnalité du programme et cela n'affiche rien dans le Serial Plotter car comme on peut mettre plusieurs signaux à la suite séparés d'une virgule ou d'un double point, l'espace est pris en compte comme un signal et le programme est impacté.

```
SPI.beginTransaction(SPISettings(8000, MSBFIRST, SPI_MODE2));
```

Cette ligne est utilisée pour configurer les paramètres de communication SPI comme la vitesse (ici cadencé à 8000 Hz) et le mode SPI (ce qui signifie que l'horloge est active en bas (Low) lorsque la transmission commence et que les données sont échantillonnées sur le front montant de l'horloge.)

Autre problème j'obtenais des valeurs (ci-dessous) non logiques dans la data :

-3V	----	>	3	625	056
-2V	----	>	2	425	056
-1V	----	>	1	200	000
0V	----	>	16	764	464
1V	----	>	15	552	160
2V	----	>	14	339	280
3V	----	>	13	125	840

□

C'est en fait les bits des 3 octets qui sont décalés, j'avais un nombre décalé de 4 bits vers la gauche. Les bits sont décalés car je lis 24 bits or **l'ADC n'en produit que 20**.

```
Serial.println( (highByte * 65536) | (midByte * 256) | lowByte );
Serial.println (result);
return result;
```

Je décalais ici de 8 bits vers la gauche l'octet du milieu, et des 16 bits l'octet de point fort. Je prenais les 8 bits de l'octet de point faible, ce qui donnait 24 bits. C'est pour cela que j'obtenais des valeurs incohérentes.

Pour résoudre ce problème, j'ai dû enlever (faire un shift) les 4 bits de poids faible (barré en rouge sur l'image ci-dessous) du dernier octet (lowbyte). Puis, j'ai décalé l'octet du milieu de 4 bits (2 puissance 4) et l'octet de poids fort de 11 bits (2 puissance 11).

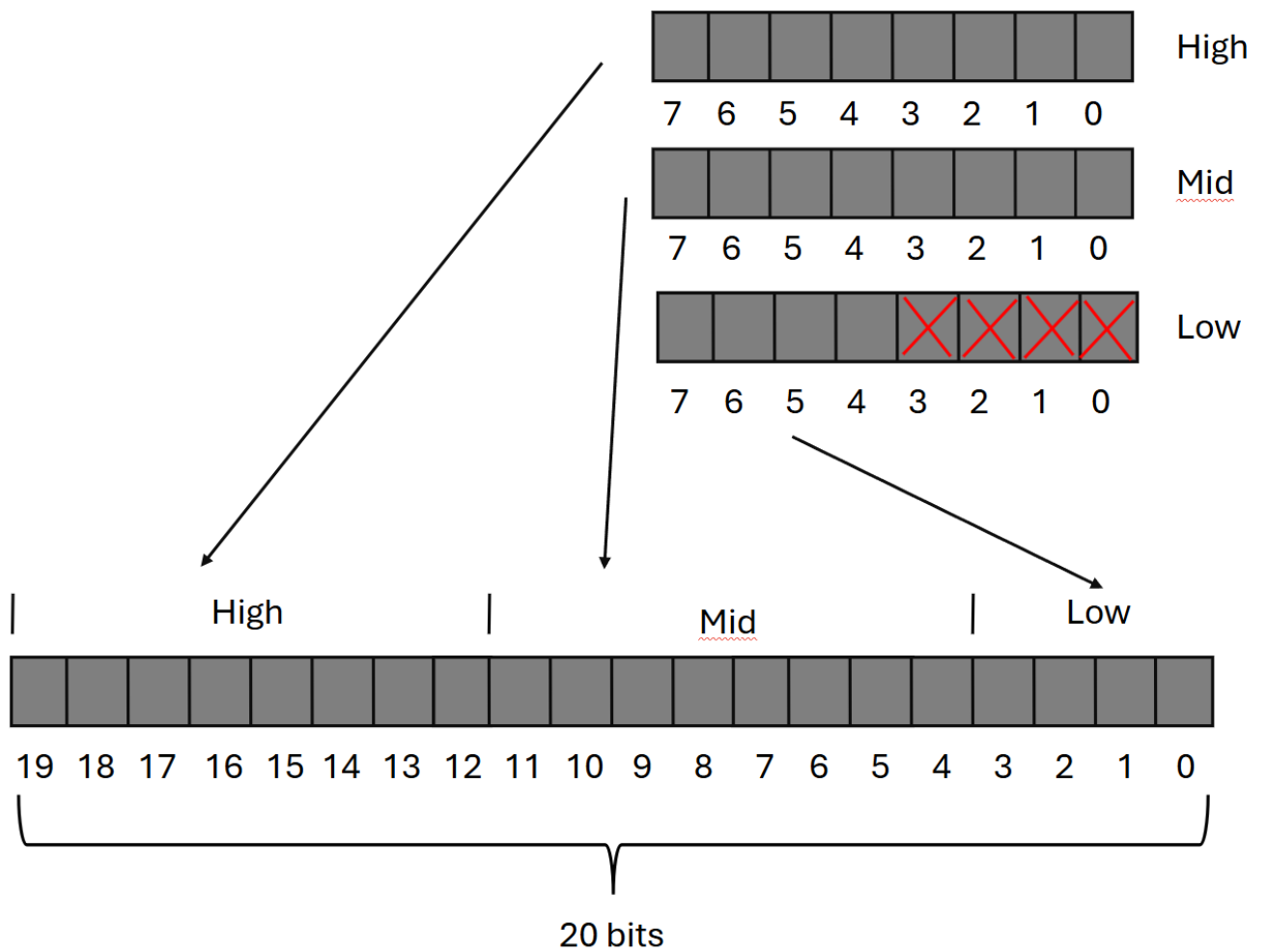


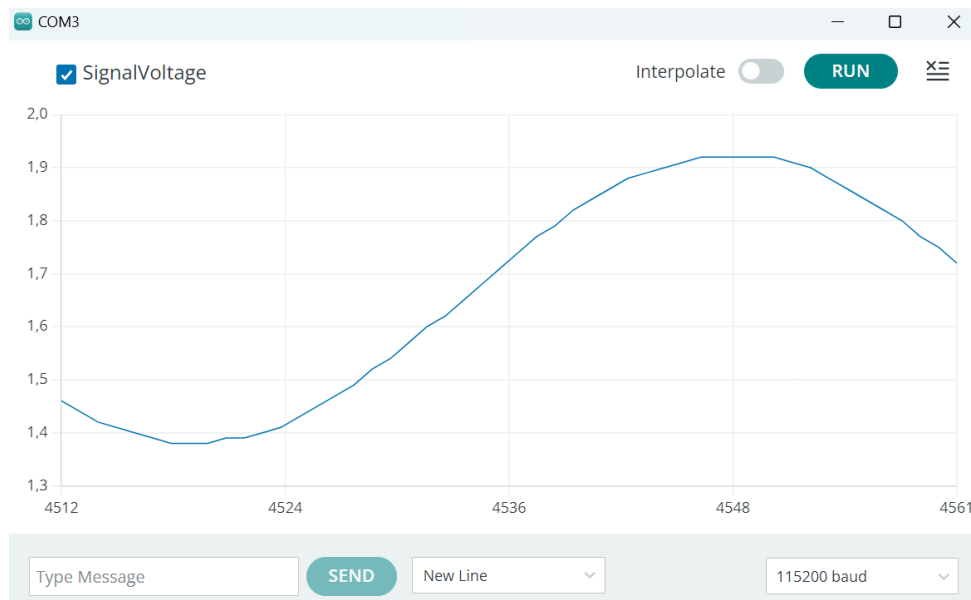
Schéma explicatif de mon erreur

Voici le code modifié après cette erreur :

```
// assemble les trois octets dans un mot de 32-bit
result = ((char)(highByte))*4096L | (midByte<<4) | (lowByte>>4);
// maintenant, result est un nombre signé de 32-bit
```

Résultat obtenu :

Malgré ces erreurs, je suis allé au bout de l'objectif de cette partie. Voici (ci-dessous) le signal retranscrit dans le serial plotter ainsi que le sérial monitor avec les valeurs du signal. J'ai pu tester différents signaux (sinus, triangle, carré...) sur le générateur et le signal est bien retranscrit en fonction du nombre d'échantillons choisi.



Capture du signal du serial plotter

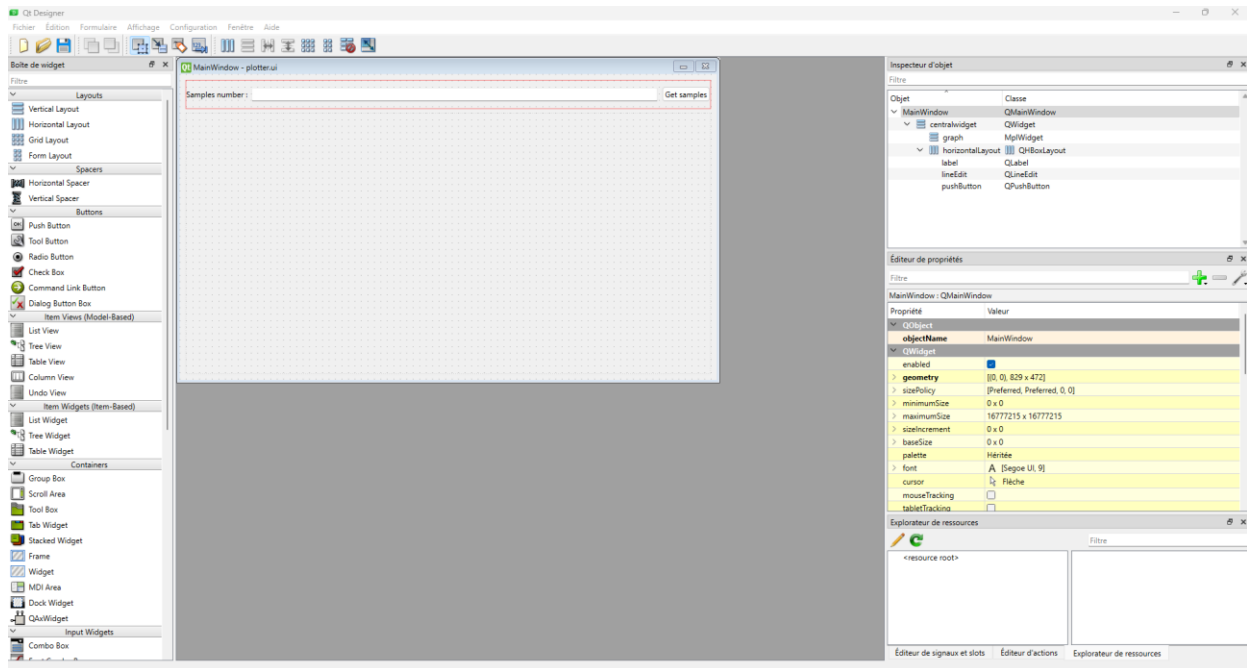
```
SignalVoltage:1.47 SignalVoltage:1.50
SignalVoltage:1.45 SignalVoltage:1.53
SignalVoltage:1.43 SignalVoltage:1.55
SignalVoltage:1.42 SignalVoltage:1.58
SignalVoltage:1.40 SignalVoltage:1.61
SignalVoltage:1.39 SignalVoltage:1.64
SignalVoltage:1.39 SignalVoltage:1.67
SignalVoltage:1.38 SignalVoltage:1.70
SignalVoltage:1.38 SignalVoltage:1.72
SignalVoltage:1.38 SignalVoltage:1.75
SignalVoltage:1.39 SignalVoltage:1.78
SignalVoltage:1.40 SignalVoltage:1.80
SignalVoltage:1.41 SignalVoltage:1.83
SignalVoltage:1.42 SignalVoltage:1.85
SignalVoltage:1.42 SignalVoltage:1.87
SignalVoltage:1.44 SignalVoltage:1.88
SignalVoltage:1.46 SignalVoltage:1.90
SignalVoltage:1.48 SignalVoltage:1.91
```

Capture des valeurs du serial monitor

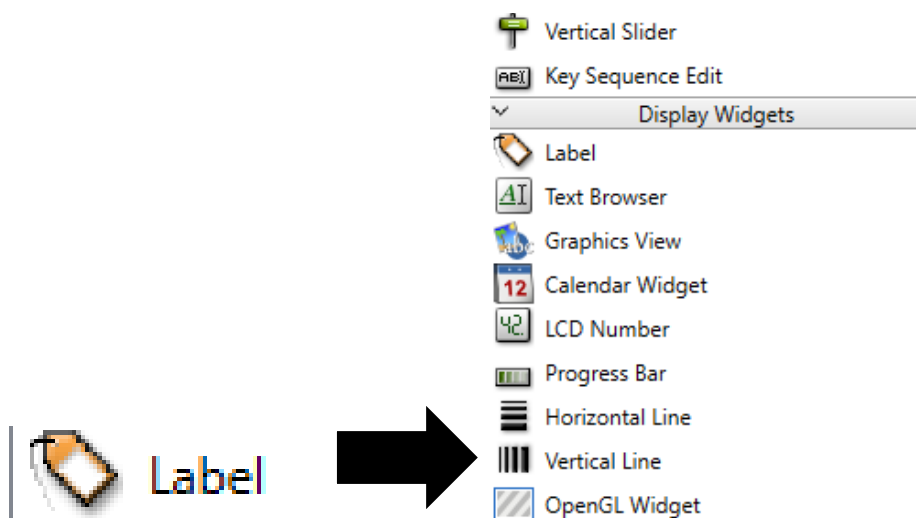
3.4 Création de l'interface sur PyQt 6

Je passe ensuite sur la partie du **design** de l'interface. J'ai fait simple, une interface avec un texte « *Samples number* » pour décrire le champ de texte/saisie la barre ou rentrer le nombre d'échantillons.

Voici l'interface PyQt Designer :



J'ai pris un « Label » sur le logiciel, dans les widgets, pour placer le texte dedans.



J'ai aussi pris un « *Line Edit* », une barre de saisie.



Line Edit

J'ai donc placé le champ de texte/saisie afin d'entrer le nombre d'échantillons que l'utilisateur souhaite.

J'ai pris un « *Push Button* », le bouton qui va permettre de valider la commande d'échantillons.



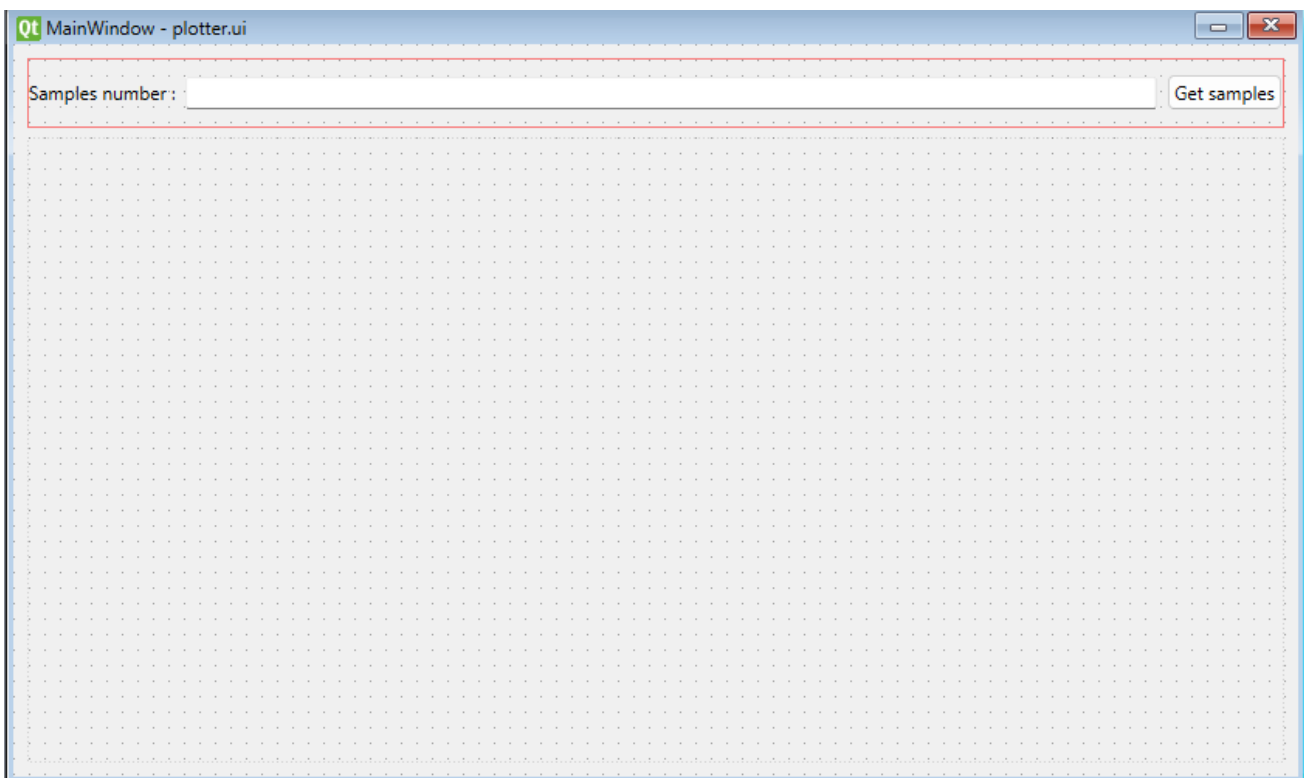
Push Button

J'ai placé un « *Graphic View* », qui va permettre de placer le graphique qui contient le signal



Graphics View

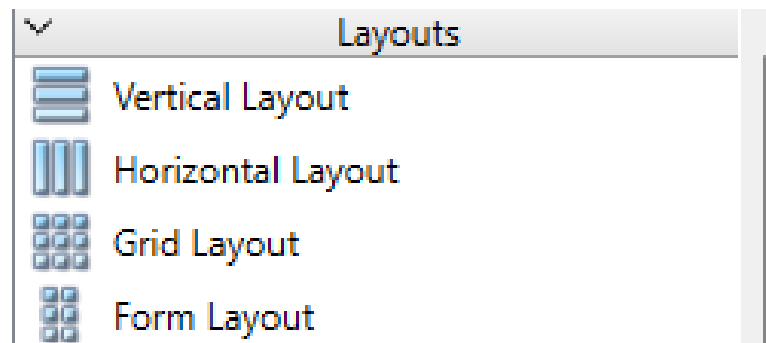
J'ai ainsi disposé les éléments de cette façon grâce aux layouts :



Le principe de layout dans Qt Designer permet **d'organiser** automatiquement les widgets dans une interface graphique de manière flexible et dynamique. En utilisant différents types de layouts tels que QHBoxLayout (disposition horizontale), QVBoxLayout (disposition verticale), QGridLayout (grille), ou QFormLayout (formulaire), les widgets s'ajustent automatiquement en fonction de la taille de la fenêtre, offrant une

interface réactive.

Cela simplifie la gestion de l'agencement et garantit que l'interface reste cohérente et adaptable sur différentes résolutions et tailles d'écrans, sans nécessiter un positionnement manuel complexe.



Ici j'ai utilisé un Horizontal Layout pour les trois widgets, de façon à les **disposer** horizontalement. J'ai utilisé aussi un Vertical Layout pour y placer le widget du graphique.

Objet	Classe
MainWindow	QMainWindow
centralwidget	QWidget
graph	MplWidget
horizontalLayout	QHBoxLayout
label	QLabel
lineEdit	QLineEdit
pushButton	QPushButton

Dans le *layoutStretch*, j'ai mis 10,90 afin que le graphique prenne 90% de la fenêtre de l'interface et que la barre de saisie prenne 10%, de façon à laisser beaucoup de place pour afficher le signal.

Layout	
layoutName	verticalLayout
layoutLeftMargin	9
layoutTopMargin	9
layoutRightMargin	AlignementGauche, AlignementCentreV
layoutBottomMargin	9
layoutSpacing	6
layoutStretch	10,90
layoutSizeConstraint	SetDefaultConstraint

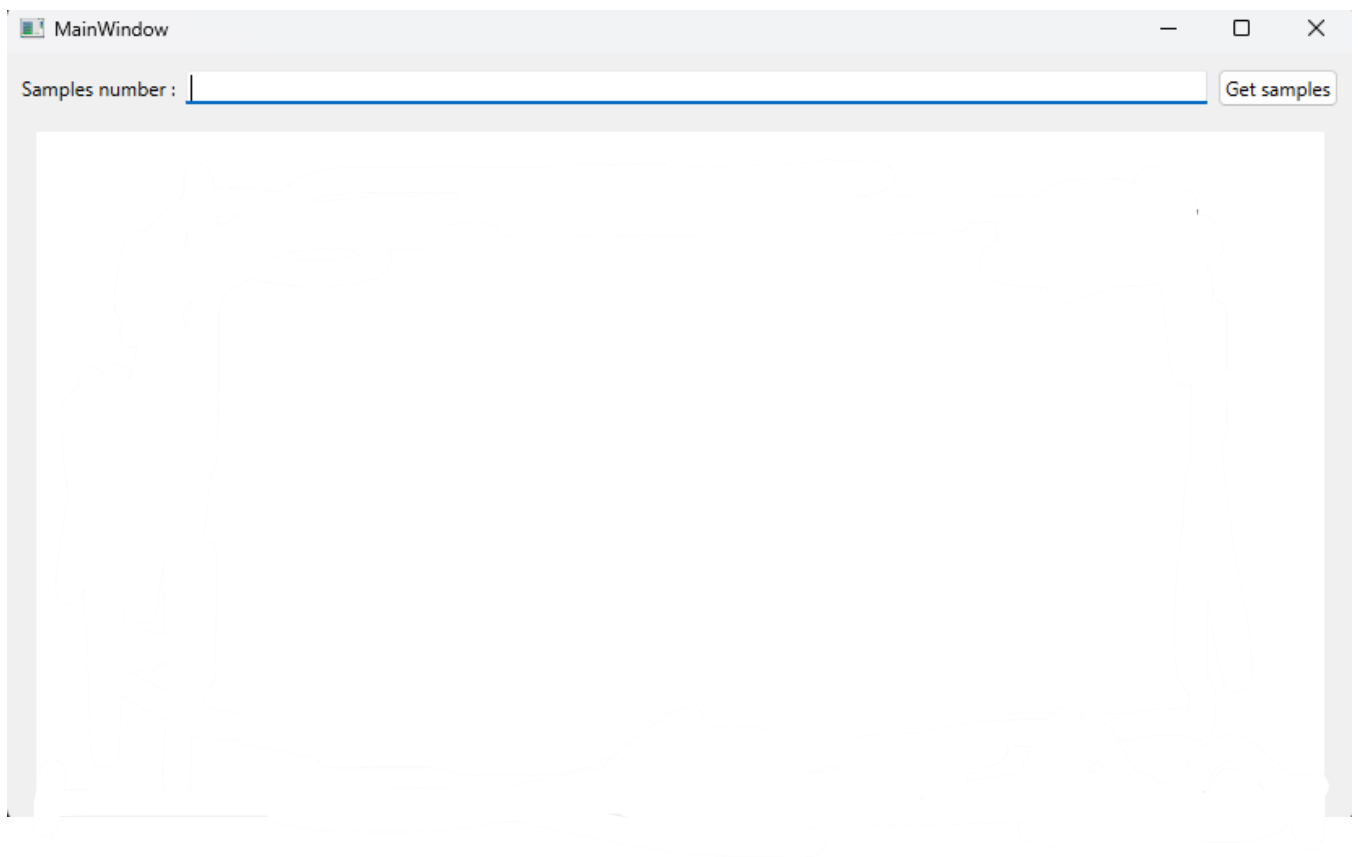
Une fois avoir fait ceci, j'ai le fichier.ui (format de PyQt designer) que je dois convertir en fichier .py (python) car le code généré contient une classe qui crée l'interface graphique définie dans le fichier ui. C'est pourquoi il est nécessaire d'utiliser un **héritage** dans le fichier Python généré et de créer un autre fichier python (explication dans la partie suivante) qui contient une nouvelle classe qui hérite de cette classe générée, afin de personnaliser l'interface.

Voici la commande à faire sur CMD du pc pour convertir le fichier

```
pyuic6 -o votre_fichier.py votre_fichier.ui
```

Résultat obtenu :

J'obtiens ce résultat :



3.5 Conception du code Python

Voici l'algorithme simplifié qui permet la compréhension de l'utilité du code Python

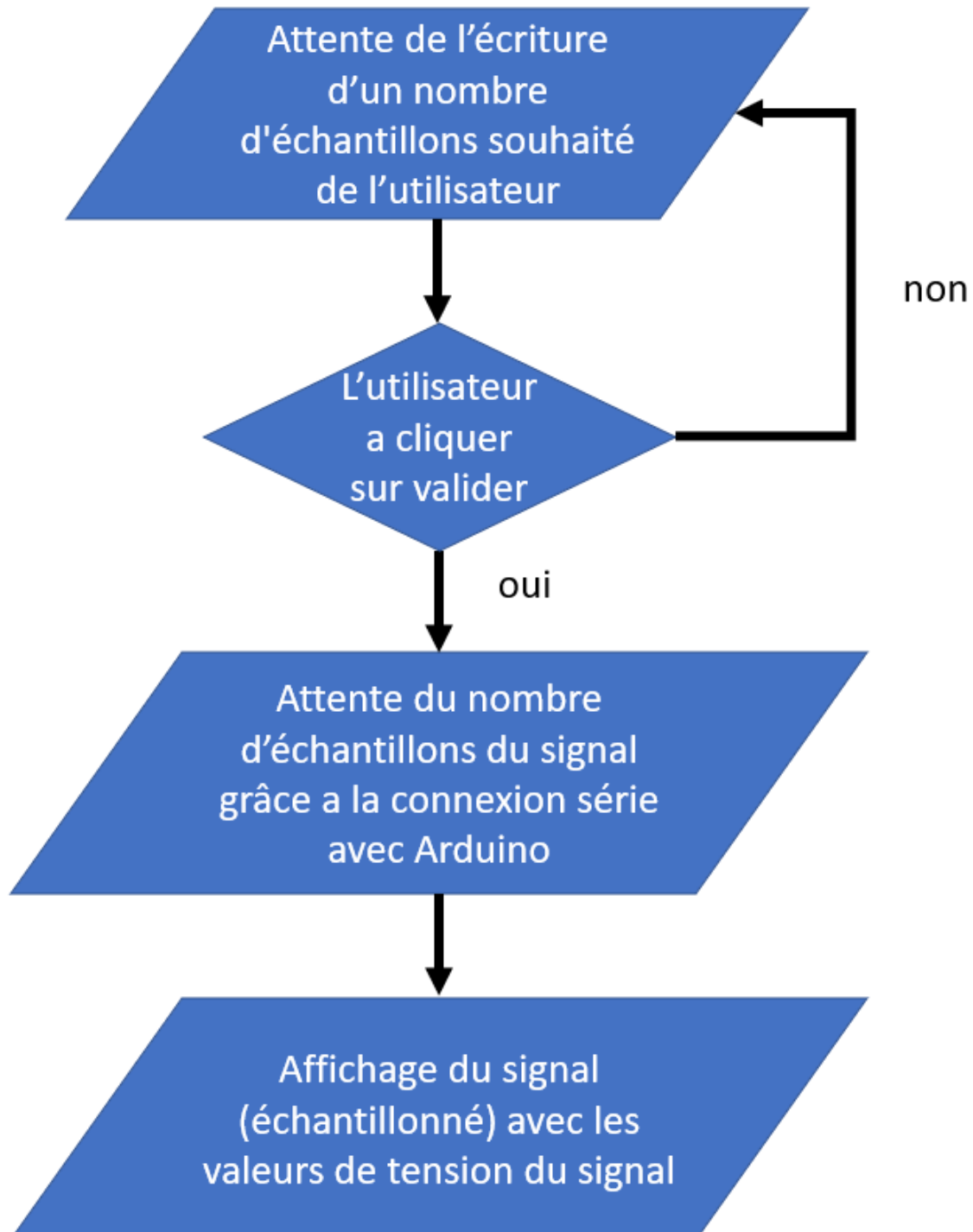


Schéma – algorithme

Pour concevoir ce code, Il faut avoir créer l'interface graphique avec un champ de texte, un bouton, et un graphique ainsi que limiter les valeurs du champ de texte (1-8191) afin de ne pas dépasser la valeur maximum que la ram du pc puisse stocker au préalable.

Dans ce code je dois :

- Lire le nombre d'échantillons à acquérir et envoyer la commande avec ce nombre d'échantillons à Arduino via la communication série lorsque le bouton « valider » est cliqué.
- Ouvrir une connexion série à Arduino pour envoyer et recevoir des données et lire les données envoyées par Arduino.
- Lire les données reçues depuis Arduino les convertir les valeurs reçues en tension (ou autres unités).
- Utiliser un timer pour mettre à jour régulièrement le graphique, le réactualiser avec les nouvelles données (x : échantillons, y : valeurs de tension).
- Afficher graphiquement les valeurs de tension en fonction du nombre d'échantillons acquis.

Voici l'explication du code final qui permet de lancer l'interface sans problème.

Tout d'abord, il faut importer les bibliothèques utiles pour créer cette interface graphique tel que :

- la bibliothèque *serial*, qui est nécessaire pour la communication avec les appareils via le port série.
- la bibliothèque *numpy* pour la manipulation de tableaux multidimensionnels et pour effectuer des calculs scientifiques et mathématiques et permet à python d'être plus efficace pour travailler avec des données numériques.
- les bibliothèques de PyQt (QApplication, QMainWindow, QIntValidator, QTimer) qui servent à ... ainsi que valider les entrées numériques de l'utilisateur et à mettre à jour l'interface à intervalles réguliers.

Le fichier *Ui_MainWindow* est une classe générée à partir d'un fichier .ui créé avec PyQt Designer. Cela permet de retrouver cette classe.

```
import sys
import serial
import threading

import numpy as np
from PyQt6.QtWidgets import QApplication, QMainWindow
from PyQt6.QtGui import QIntValidator
from PyQt6.QtCore import QTimer

from plotter import Ui_MainWindow
```

Ensuite, je dois créer une classe *MainWindow* qui hérite à la fois de *QMainWindow* (classe de base de Qt pour une fenêtre principale d'application) et de *Ui_MainWindow* (la classe générée automatiquement par PyQt).

Voici le fichier a hérité qui contient *Ui_MainWindow* :

```

from PyQt6 import QtCore, QtGui, QtWidgets

from mplwidget import MplWidget

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        MainWindow.setObjectName("MainWindow")
        MainWindow.resize(829, 472)
        self.centralwidget = QtWidgets.QWidget(parent=MainWindow)
        self.centralwidget.setObjectName("centralwidget")
        self.verticalLayout = QtWidgets.QVBoxLayout(self.centralwidget)
        self.verticalLayout.setObjectName("verticalLayout")
        self.horizontalLayout = QtWidgets.QHBoxLayout()
        self.horizontalLayout.setObjectName("horizontalLayout")
        self.label = QtWidgets.QLabel(parent=self.centralwidget)
        self.label.setObjectName("label")
        self.horizontalLayout.addWidget(self.label)
        self.lineEdit = QtWidgets.QLineEdit(parent=self.centralwidget)
        self.lineEdit.setObjectName("lineEdit")
        self.horizontalLayout.addWidget(self.lineEdit)
        self.pushButton = QtWidgets.QPushButton(parent=self.centralwidget)
        self.pushButton.setObjectName("pushButton")
        self.horizontalLayout.addWidget(self.pushButton)
        self.verticalLayout.addLayout(self.horizontalLayout)
        self.graph = MplWidget(parent=self.centralwidget)
        self.graph.setObjectName("graph")
        #self.verticalLayout_3 = QtWidgets.QVBoxLayout(self.graph)
        #self.verticalLayout_3.setObjectName("verticalLayout_3")
        self.verticalLayout.addWidget(self.graph)
        self.verticalLayout.setStretch(0, 10)
        self.verticalLayout.setStretch(1, 90)
        MainWindow.setCentralWidget(self.centralwidget)

        self.retranslateUi(MainWindow)
        QtCore.QMetaObject.connectSlotsByName(MainWindow)

    def retranslateUi(self, MainWindow):
        _translate = QtCore.QCoreApplication.translate
        MainWindow.setWindowTitle(_translate("MainWindow", "MainWindow"))
        self.label.setText(_translate("MainWindow", "Samples number :"))
        self.pushButton.setText(_translate("MainWindow", "Get samples"))

```

Afin de limiter les valeurs que l'utilisateur peut entrer je crée un validateur pour le champ de texte `lineEdit`. Le validateur `QIntValidator` restreint les entrées à des nombres entiers compris entre 1 et 8191. Cela permet surtout d'éviter que la RAM du pc soit pleine car le pc que j'utilise est limité à stocker 8191 échantillons du signal.

Je crée un objet de validation QIntValidator qui permet de restreindre les valeurs à des entiers dans une plage spécifiée (ici, entre 1 et 8191). Ce validateur va être appliqué à un champ de texte pour garantir que l'utilisateur n'entre que des valeurs numériques valides dans cette plage.

J'écris cette ligne pour lier l'action de cliquer sur le bouton (pushButton) à la fonction on_pushButton_clicked.

```
# Configuration du port série
SERIAL_PORT = "COM3"
BAUD_RATE = 115200

class MainWindow(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        # Limiter les chiffres (1-8191)
        validator = QIntValidator(1, 8191)
        self.lineEdit.setValidator(validator)

        self.pushButton.clicked.connect(self.on_pushButton_clicked) # get_samples_btn, on_get_samples_btn_clicked
```

Je définis deux listes vides qui seront utilisées pour stocker les données à afficher sur le graphique, une qui contiendra les valeurs pour l'axe des X (le nombre d'échantillons) et une contiendra les valeurs pour l'axe des Y (les valeurs de tension).

```
# Initialisation des données pour le graphique
self.x_data = []
self.y_data = []
```

Je configure la communication en série avec la carte Arduino en définissant la vitesse de transmission des données (le Baud-rate) et le port série. Le timeout signifie que la connexion série attendra 5 secondes avant d'abandonner si aucune donnée n'est reçue.

thread.self.serial_thread.daemon = True Cela rend le thread un thread dit "daemon", ce qui signifie qu'il s'arrêtera automatiquement lorsque l'application principale se ferme, sans avoir besoin d'être arrêté. Cela permet aussi à l'interface d'être exécutée en arrière-plan.

```
# Initialisation de la communication série et lecture en continue
self.serial_port = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=5.0)
self.serial_thread = threading.Thread(target=self.read_serial_data)
self.serial_thread.daemon = True
self.serial_thread.start()
```

J'utilise un timer pour déclencher des actions à intervalles réguliers. C'est-à-dire que dès que le timer atteint son délai, il y a un self.uptade sur le graphique. Cela signifie que l'interface graphique sera mise à jour.

```
# Mise à jour du graphique tous les 500ms
self.timer = QTimer(self)
self.timer.timeout.connect(self.update_graph)
self.timer.start(500)
```

On fait appel à cette méthode quand l'utilisateur appuie sur le bouton pour valider la commande d'échantillons.

Cette fonction récupère le texte du champ `lineEdit` (qui contient le nombre d'échantillons à acquérir), puis envoie cette valeur à l'Arduino via la communication série.

```
def on_pushButton_clicked(self):
    N = self.lineEdit.text()
    self.serial_port.write(f"samples {N}".encode('utf-8'))
```

Ici je vérifie qu'il y ait des données disponibles (`in_waiting` vérifie cela), si les données sont un nombre valide, elles sont converties en une valeur de tension (suivant une formule spécifique), puis ajoutées aux listes `y_data` (valeurs de tension) et `x_data` (indices des échantillons). Ces valeurs seront ensuite utilisées pour le graphique.

```
def read_serial_data(self):
    while True:
        if self.serial_port.in_waiting > 0:
            data = self.serial_port.readline().decode('utf-8').strip()
            if data.lstrip('-').isdigit():
                tension = -(int(data) + 920) / 75_742 ## référencer la formule
                self.y_data.append(tension)
                self.x_data.append(len(self.y_data))
```

Comme je l'ai expliqué avant, cette méthode est appelée périodiquement (via un `QTimer`) pour mettre à jour le graphique. Cela efface les axes précédents du graphique avec un `clear`, puis trace les nouvelles données. Cela définit également un titre, des étiquettes pour les axes X et Y, et ajoute une légende. Enfin, le graphique est redessiné avec `draw` (à chaque fois que le timer redemarre).

```
def update_graph(self):
    # Mise à jour des données du graphique
    self.graph.canvas.axes.clear()
    self.graph.canvas.axes.plot(self.x_data[1:], self.y_data[1:], label='Data')
    self.graph.canvas.axes.set_title("Signal échantillonné")
    self.graph.canvas.axes.set_xlabel("Samples")
    self.graph.canvas.axes.set_ylabel("Tension (V)")
    self.graph.canvas.axes.legend()
    self.graph.canvas.draw()
```

Ce bloc de code est l'entrée principale de l'application. Lorsque le programme est exécuté, il crée une instance de l'application Qt (`QApplication`), crée la fenêtre principale (`MainWindow`), l'affiche à l'écran avec `window.show()`, et démarre la boucle d'événements de l'application avec `app.exec()`. Le programme se termine proprement en appelant `sys.exit()` lorsque l'application se ferme.

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```


Quelques erreurs rencontrées :

J'ai eu un problème (dans cette partie ci-dessous) avec le `is.isdigit` qui vérifie si tous les caractères de la chaîne de caractères (ici la data) sont des chiffres arabes de 0 à 9 (n'accepte pas les lettres les nombres décimaux) nombres qui ne regroupe pas tous les nombres naturels.

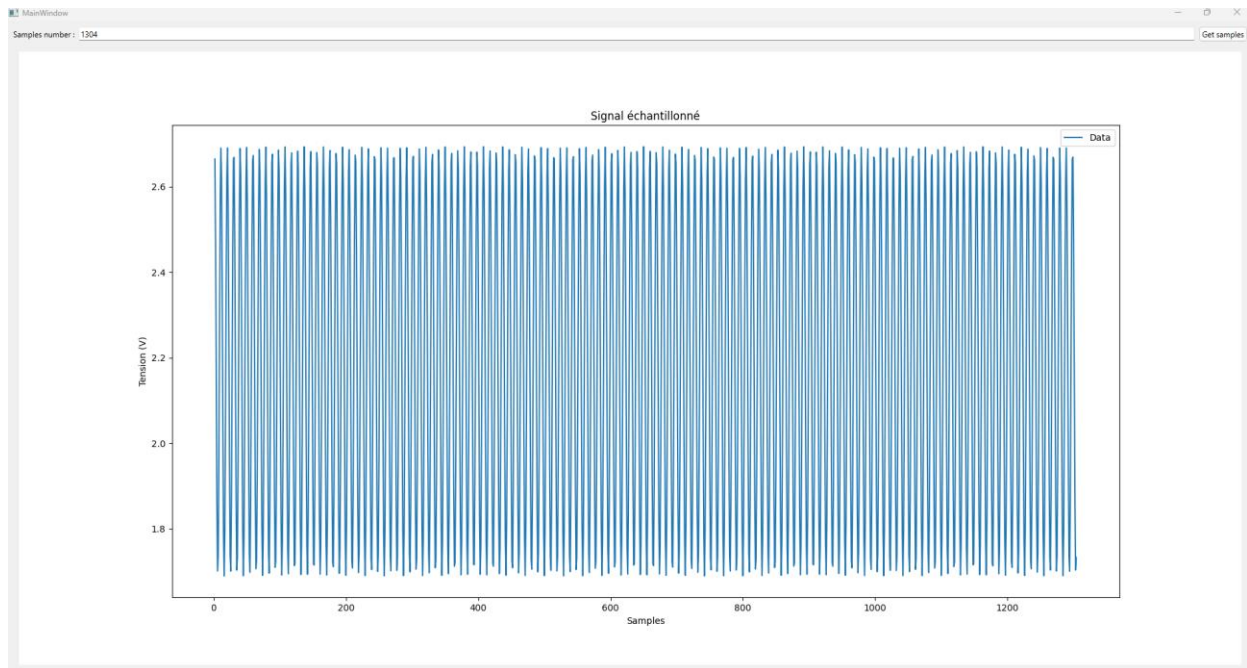
```
def read_serial_data(self):
    while True:
        if self.serial_port.in_waiting > 0:
            data = self.serial_port.readline().decode('utf-8').strip()
            if data.isdigit():
                tension = -(int(data) + 920) / 75_742 ## référencer la formule
                self.y_data.append(tension)
                self.x_data.append(len(self.y_data))
```

J'ai donc dû ajouter un `.lstrip('-')` afin de supprimer le signe négatif (-) au début de la chaîne (si présent), pour pouvoir vérifier si le reste de la chaîne est numérique, si la chaîne restante contient uniquement des chiffres, cela signifie que data représente un nombre entier valide, et le programme peut alors le traiter.

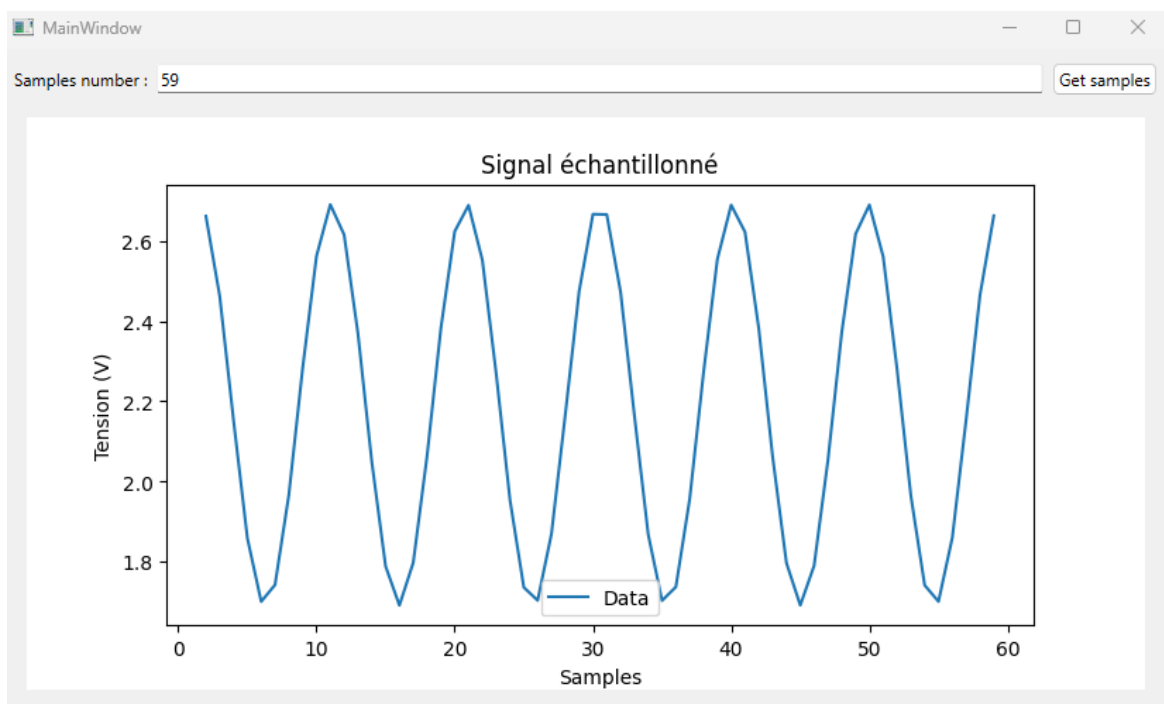
```
if data.lstrip('-').isdigit():
```

Résultat obtenu de l'interface graphique et projection

L'interface graphique fonctionne bien, échantillonne bien le signal selon la commande de l'utilisateur et répond aux attentes de l'utilisateur.



Capture de l'interface avec un nombre d'échantillons de 1304



Capture de l'interface avec un nombre d'échantillons de 59

Avec plus de temps, j'aurais pu ajouter un bouton **reset** qui permet de réinitialiser le graphique, afin que l'utilisateur puisse facilement revenir à l'état initial du graphique sans avoir à redémarrer l'application. Un bouton **stop** aurait également été utile pour permettre à l'utilisateur d'interrompre l'échantillonnage des données à tout moment, ce qui est particulièrement important lorsqu'il souhaite analyser un jeu de données sans qu'il continue à se mettre à jour ou pour arrêter un échantillonnage en cours qui pourrait durer trop longtemps.

De plus, pour améliorer l'expérience utilisateur, je mettrai en place une barre de **scroll horizontale et verticale** afin que l'utilisateur puisse naviguer facilement dans la fenêtre du graphique, en particulier pour les signaux non continus où la visualisation peut s'étendre sur un grand nombre de points de données. Cela lui permettrait de zoomer sur différentes sections du graphique et d'avoir une meilleure vue d'ensemble des tendances ou des anomalies dans les données.

Ensuite, j'ajouterais des **curseurs horizontaux et verticaux** permettant à l'utilisateur de mesurer plus précisément les valeurs sur le graphique. Ces curseurs pourraient être déplacés pour afficher les coordonnées exactes des points sur le graphique, ce qui serait particulièrement utile pour une analyse détaillée. De plus, pour rendre cette fonctionnalité encore plus pratique, j'aurais placé un **petit tableau sous le graphique** où les valeurs des coordonnées des curseurs, ainsi que d'autres informations pertinentes comme la fréquence ou l'amplitude, seraient affichées en temps réel. Cela permettrait à l'utilisateur de voir instantanément les valeurs sans avoir à chercher manuellement sur le graphique, offrant ainsi une manière plus intuitive et rapide de travailler avec les données.

Enfin, il serait également intéressant d'intégrer un **système de zoom interactif** sur le graphique, afin que l'utilisateur puisse zoomer à la fois horizontalement et verticalement pour examiner des détails plus fins, tout en gardant une bonne vue d'ensemble. Ce type de fonctionnalité rendrait l'outil encore plus puissant et accessible pour les utilisateurs de tous niveaux.

Conclusion

Au terme de ce stage au sein du LIRA, j'ai eu l'opportunité de développer une **interface graphique** en Python pour un utilisateur afin de recueillir et de **visualiser les échantillons numériques** dérivés d'un signal analogique en temps réel. Lors de ce stage de huit semaines, j'ai pu mettre en pratique mes connaissances théoriques acquises durant ma formation.

Ce stage a été très enrichissant, car il m'a permis d'approfondir le domaine de l'informatique. Il m'a permis de développer mes compétences en programmation-objet. J'ai également eu l'occasion d'utiliser PyQt Designer pour la première fois afin de concevoir une interface graphique intuitive pour le projet, ce qui m'a permis d'approfondir mes compétences en développement d'interfaces utilisateurs.

J'ai eu l'opportunité de participer à un projet de R&D impliquant des acteurs de domaines d'expertise différents. Ce stage m'a finalement aussi permis de découvrir le métier d'ingénieur développement logiciel. Je peux désormais dire que ce métier pourrait tout à fait me plaire dans ma future carrière. Cette expérience a contribué à approfondir ma connaissance des **systèmes embarqués** et à souligner l'importance des interfaces de communication dans la gestion des données électroniques.

La première partie du stage traitait les connaissances en Python ainsi que l'acquisition des données sur Arduino. Une fois cette acquisition faite, je suis passé à la conception de l'interface. Finalement, l'interface a été développée permettant aux utilisateurs et aux scientifiques d'observer les convertisseurs analogiques numériques fonctionnent toujours bien.

L'objectif a été réalisé avec succès, et cette expérience m'a fourni une base solide pour entreprendre des projets futurs dans le secteur de l'électronique et de l'informatique.

Annexes

- Extraits de la documentation technique de l'ADC LTC237



LTC2370-16

16-Bit, 2Msps, Pseudo-Differential Unipolar SAR ADC with 94dB SNR

FEATURES

- 2Msps Throughput Rate
- $\pm 0.85\text{LSB}$ INL (Max)
- Guaranteed 16-Bit No Missing Codes
- Low Power: 19mW at 2Msps, 19 μW at 2ksps
- 94dB SNR (Typ) at $f_{\text{IN}} = 2\text{kHz}$
- -112dB THD (Typ) at $f_{\text{IN}} = 2\text{kHz}$
- Guaranteed Operation to 125°C
- 2.5V Supply
- Pseudo-Differential Unipolar Input Range: 0V to V_{REF}
- V_{REF} Input Range from 2.5V to 5.1V
- No Pipeline Delay, No Cycle Latency
- 1.8V to 5V I/O Voltages
- SPI-Compatible Serial I/O with Daisy-Chain Mode
- Internal Conversion Clock
- 16-Lead MSOP and 4mm $\times$ 3mm DFN Packages

APPLICATIONS

- Medical Imaging
- High Speed Data Acquisition
- Portable or Compact Instrumentation
- Industrial Process Control
- Low Power Battery-Operated Instrumentation
- ATE

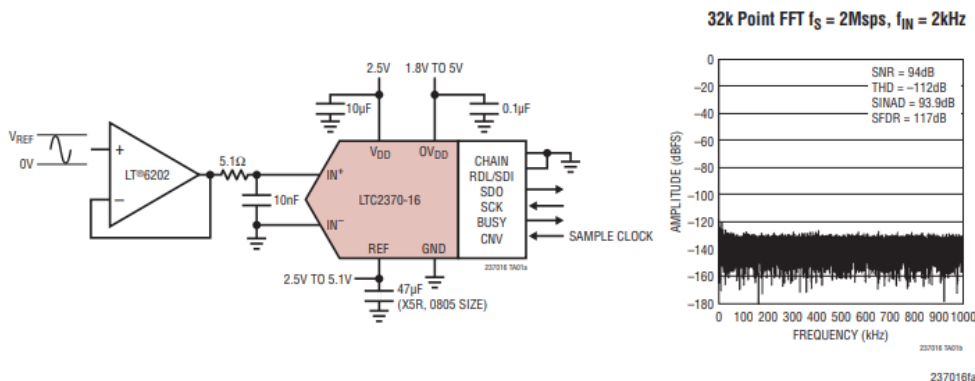
DESCRIPTION

The LTC[®]2370-16 is a low noise, low power, high speed 16-bit successive approximation register (SAR) ADC. Operating from a 2.5V supply, the LTC2370-16 has a 0V to V_{REF} pseudo-differential unipolar input range with V_{REF} ranging from 2.5V to 5.1V. The LTC2370-16 consumes only 19mW and achieves $\pm 0.85\text{LSB}$ INL maximum, no missing codes at 16 bits with 94dB SNR.

The LTC2370-16 has a high speed SPI-compatible serial interface that supports 1.8V, 2.5V, 3.3V and 5V logic while also featuring a daisy-chain mode. The fast 2Msps throughput with no cycle latency makes the LTC2370-16 ideally suited for a wide variety of high speed applications. An internal oscillator sets the conversion time, easing external timing considerations. The LTC2370-16 automatically powers down between conversions, leading to reduced power dissipation that scales with the sampling rate.

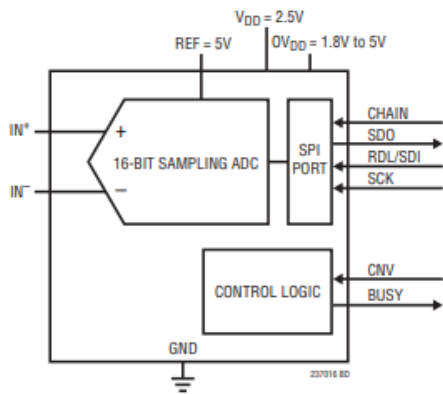
LT, LT, LTC, LTM, Linear Technology and the Linear logo are registered trademarks and SoftSpan is a trademark of Linear Technology Corporation. All other trademarks are the property of their respective owners. Protected by U.S. Patents, including 7705765.

TYPICAL APPLICATION



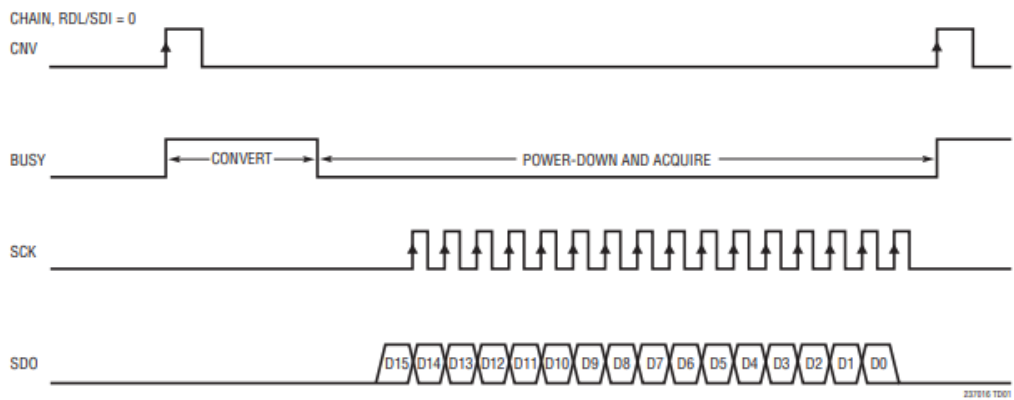
1

FUNCTIONAL BLOCK DIAGRAM



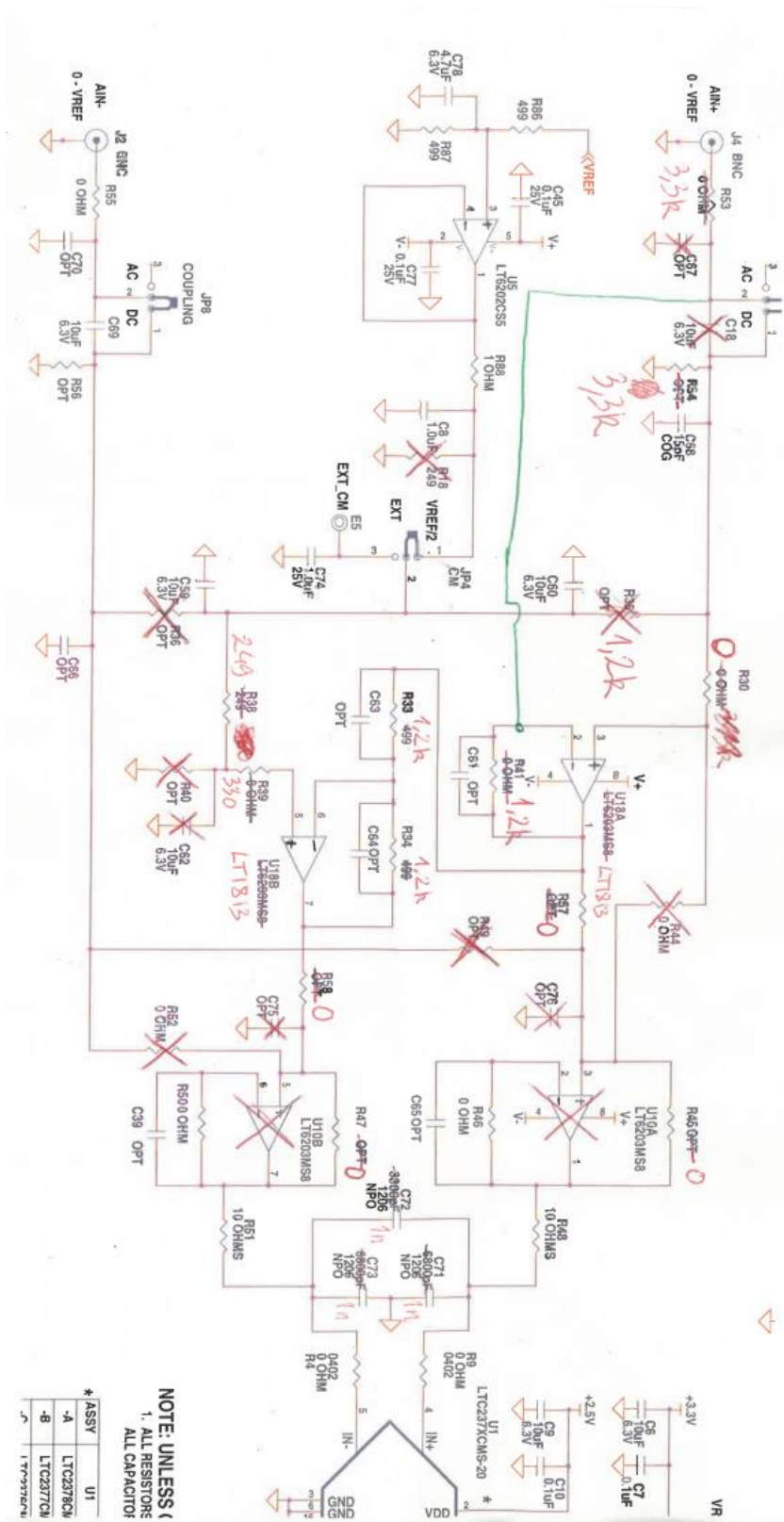
TIMING DIAGRAM

Conversion Timing Using the Serial Interface



237016fa

- **Schéma du LTC237 (modifié par les ingénieurs)**

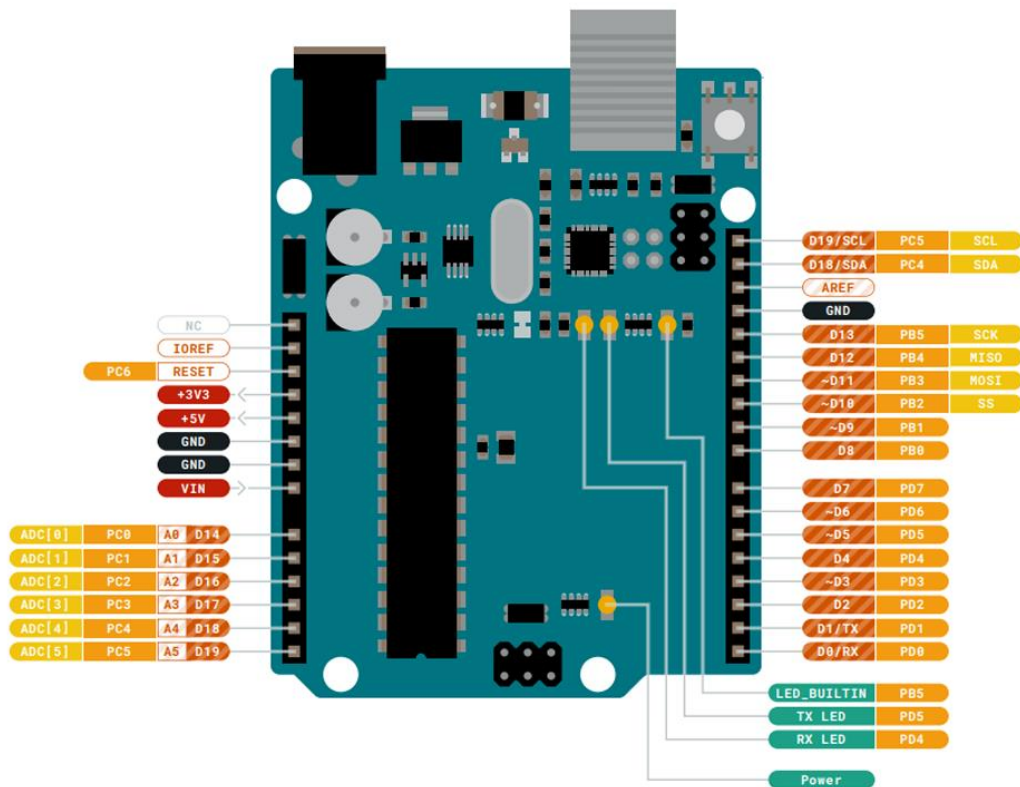


- Extrait de la documentation technique de la carte Arduino



Arduino® UNO R3

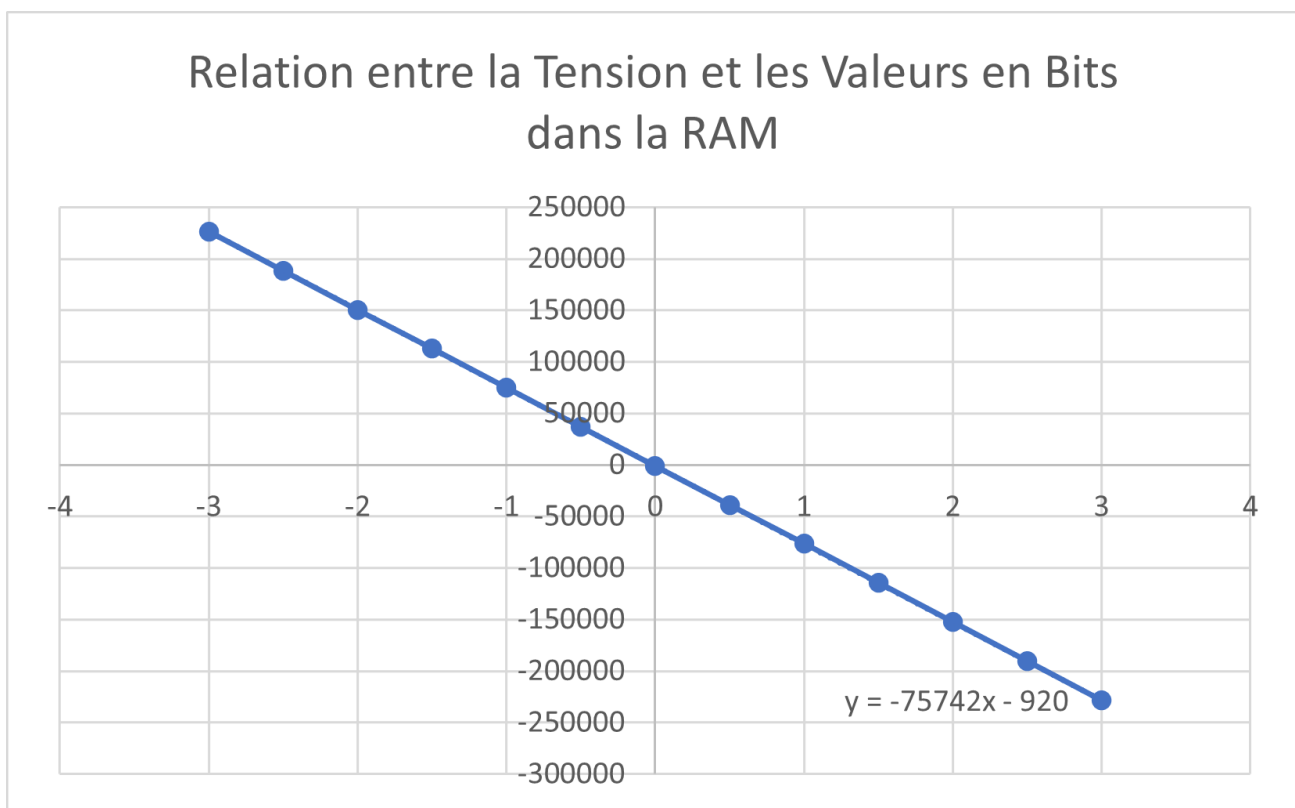
5 Connector Pinouts



Pinout

- **Calcul de la conversion de la data en tension**

Le calcul " $tension = -(int(data) + 920) / 75_742$ " ($y = ax + b$) sert à convertir une donnée brute (contenue dans la variable data) en une valeur physique, ici une tension. D'abord, la variable data est convertie en entier à l'aide de $int(data)$. Ensuite, 920 est ajouté à cette valeur, ce qui représente l'ordonnée à l'origine (b). Ce résultat est ensuite divisé par -75 742, le coefficient directeur (x). J'ai trouvé ces valeurs en faisant un tableau Excel (ci-dessous) en listant pour avec plusieurs tensions différentes la data obtenu.



Liste des abréviations et sigles employés

Abréviations/Sigles	Significations
ADC	Analog to Digital Converter
ANR	Agence Nationale de la Recherche
CNES	Centre national d'études spatiales
CNRS	Centre national de la recherche scientifique
CPU	Central Processing Unit
ESA	European Space Agency
ESO	European Southern Observatory
FFT	Fast Fourier Transform
FIFO	First In First Out
FPGA	Field-Programmable Gate Array
FTDI	Future Technology Devices International
I2C	Inter-Integrated Circuit
INSU	Institut National des Sciences de l'Univers
LIRA	Laboratoire d'Instrumentation et de Recherche en Astrophysique
LESIA	Laboratoire d'Études Spatiales et d'Instrumentation en Astrophysique
MVVM	Model View ViewModel
NASA	National Aeronautics and Space Administration
R&D	Recherche et développement
SIEL	Service d'ingénierie électronique du LESIA
SGSE	Software GSE
SPI	Serial Peripheral Interface

Bibliographie

<https://www.obspm.fr/>

https://www.sciencesetavenir.fr/espace/en-350-ans-l-observatoire-de-paris-a-marque-l-histoire-de-l-astronomie_113936

<https://www.oca.eu/>

<http://www.universalis-edu.com/encyclopedie/fondation-de-l-observatoire-de-paris/> <http://sci.esa.int/solar-orbiter/>

<http://lesia.obspm.fr/>

<https://observatoiredeparis.psl.eu/-laboratoires-de-recherche-.html>

<https://observatoiredeparis.psl.eu/>

<http://sci.esa.int/solar-orbiter/>

<https://www.cnrs.fr/fr>

<https://www.iap.fr/personnel/recherche/recherche.php>

https://www.iap.fr/vie_scientifique/seminaires/seminaires.php

<https://www.iap.fr/accueil/annuaire/annuaire.php>

Crédits iconographiques

Panorama du site de Meudon – p.5

<https://www.obspm.fr/-histoire-du-site-de-meudon-.html>

Organigramme de l’Observatoire de Paris – p.6

<https://www.obspm.fr/organigramme-de-l.html>

Organigramme général du LESIA – p.7

<http://lesia.obspm.fr/-Organisation-du-laboratoire-187-.html>

Organigramme général du SIEL – p.8

<http://lesia.obspm.fr/-Organisation-du-laboratoire-187-.html>

Budget du LESIA de 2012 à 2016 – p.9

<http://lesia.obspm.fr/-Les-budgets-.html>

Vent solaire et magnétosphère – p.10

<http://lesia.obspm.fr/-Vent-solaire-et-milieu-.html>

<http://www.encyclopedie-environnement.org/air/la-magnetosphere/>

Image du FT2232H Mini Module – p.14

Issu de la DataSheet du DS FT2232H Mini Module